LAB07 – PHÂN ĐOẠN ẢNH SỬ DỤNG NGƯỠNG

I. MỤC TIÊU

- Giải thích bản chất phân đoạn nhị phân, ảnh nhãn và sự phụ thuộc vào giá trị ngưỡng.
- Cài đặt ngưỡng cố định, ngưỡng kép/đa mức và thuật toán tìm ngưỡng toàn cục lặp bằng NumPy.
- Hiểu và cài đặt Otsu từ histogram; đối chiếu kết quả với cv2.threshold.
- Thực nghiệm Adaptive Mean/Gaussian và đánh giá tác động của blockSize, C, tiền xử lý.
- Xây dựng pipeline ứng dụng; trình bày hình ảnh trung gian, chỉ số định lượng và nhận xét có căn cứ.

II. CHUẨN BỊ VÀ QUY ĐỊNH

- Môi trường: Google Colab/Jupyter; thư viện chính: cv2, numpy, matplotlib. Không bắt buộc scikit-image.
- Ảnh đầu vào 8-bit; ưu tiên ảnh xám. Ảnh màu phải chuyển sang xám và giải thích lý do.
- Mỗi bài hiển thị ảnh gốc, histogram (khi cần), ảnh phân đoạn và thông số; notebook phải chạy Run All.
- Khi so sánh các thuật toán, dùng cùng ảnh đầu vào, cùng kích thước và cùng quy ước foreground.
- Quy ước cho bài tập: pixel > T là foreground (255), pixel ≤ T là background (0), trừ khi bài nêu rõ khác. cv2.THRESH_BINARY tuân theo quy ước này.

III. DỮ LIỆU ĐỀ XUẤT

Nguồn dữ liệu	**Ảnh gợi ý / đặc điểm**	**Bài phù hợp**	**Đường dẫn**
scikit-image sample data	camera, coins, page, text, binary_blobs; ảnh chuẩn minh họa	B1–B10	https://scikit-image.org/docs/stable/api/skimage.data.html
OpenCV samples	sudoku.png; ảnh ô số chịu ảnh hưởng ánh sáng	B7, B8, B10	https://github.com/opencv/opencv/tree/4.x/samples/data
DIBCO / H-DIBCO	Ảnh tài liệu lịch sử suy giảm, có ground truth theo từng bộ	B8–B10	https://vc.ee.duth.gr/dibco2019/
Ảnh tự chụp	Hóa đơn, trang sách, linh kiện trên nền tương phản, hạt/đồng xu	B2, B7–B10	Sinh viên tự thu thập; ghi nguồn và điều kiện chụp

Lưu ý dataset: Không phải ảnh nào cũng phù hợp cho mọi thuật toán. “coins” phù hợp tách vật thể; “page”/“sudoku” phù hợp ngưỡng thích nghi; “camera” có thể minh họa thất bại của ngưỡng toàn cục. DIBCO phù hợp bài nâng cao và cần tải ảnh cùng ground truth từ nguồn tương ứng. Nếu không truy cập được nguồn, sử dụng ảnh tự chụp theo tiêu chí mô tả.

IV. NỘI DUNG BÀI THỰC HÀNH

Bài 1. Phân đoạn ngưỡng cố định trên ma trận (cơ bản)

Cho I = [[19,23,37,43,43],[10,19,23,37,43],[10,10,19,23,37],[10,10,10,19,23],[9,10,10,10,19]]. Đây là ma trận từ bài tập trang 63 của bài giảng.

- Viết hàm threshold_manual(I,T) dùng NumPy, không gọi cv2.threshold; T=19 và T=23.
- In hai ma trận nhị phân (0/255); tính số pixel foreground và tỷ lệ foreground (%).
- Đối chiếu với cv2.threshold và xác nhận hai kết quả trùng nhau hoàn toàn.
- Giải thích cách xử lý pixel có giá trị đúng bằng T.

```
def threshold_manual(img, T):
  # TODO: trả về ảnh uint8 chỉ chứa 0 và 255
  pass


I = np.array([[19,23,37,43,43], [10,19,23,37,43],
        [10,10,19,23,37], [10,10,10,19,23],
        [9,10,10,10,19]], dtype=np.uint8)
# TODO: thử T=19, 23 và kiểm chứng bằng cv2.threshold
```

Bài 2. Khảo sát ảnh thực và lựa chọn ngưỡng (cơ bản)

- Chọn ảnh coins hoặc ảnh vật thể tối/sáng trên nền tương phản; đọc ảnh xám.
- Vẽ histogram 256 mức và thử T = 80, 120, 160 (được phép điều chỉnh nếu ảnh khác).
- Hiển thị 1 ảnh gốc + 3 mask; lập bảng T, số pixel trắng, tỷ lệ pixel trắng.
- Chọn T hợp lý, chỉ ra ít nhất hai vùng bị mất hoặc bị nhận nhầm nếu T quá thấp/quá cao.

Bài 3. So sánh năm kiểu threshold trong OpenCV

- Áp dụng THRESH_BINARY, BINARY_INV, TRUNC, TOZERO, TOZERO_INV với T=127.
- Vẽ kết quả 2×3 và bảng giải thích công thức đầu ra của mỗi kiểu.
- Kiểm chứng đầu ra tại các giá trị pixel 80, 127, 180; nêu kiểu nào không tạo mask nhị phân.

Bài 4. Ngưỡng kép và ngưỡng đa mức

- Tạo hàm double_threshold(I,T1,T2) trả về 255 nếu $T1 < I \leq T2$, còn lại 0.
- Tạo hàm multilevel(I,T1,T2) trả về 0,127,255 tương ứng ba khoảng $I \leq T1$; $T1 < I \leq T2$; $I > T2$.
- Dùng T1=60,T2=150 trên ảnh có tối thiểu ba nhóm cường độ; vẽ histogram kèm hai đường ngưỡng.
- Thống kê số pixel từng lớp; giải thích khác biệt giữa ảnh mask nhị phân và ảnh nhãn 3 mức.

```
def multilevel(img, T1, T2):
  assert 0 <= T1 < T2 <= 255
  out = np.zeros(img.shape, dtype=np.uint8)
  # TODO: gán lớp giữa = 127 và lớp trên = 255
  return out
```

Bài 5. Tìm ngưỡng toàn cục lặp

Theo trang 46–47: T0=(Imin+Imax)/2; chia G1={I≤T}, G2={I>T}; cập nhật Tnew=(μ1+μ2)/2 đến khi $|Tnew - T| < \varepsilon$.

- Cài đặt iterative_threshold(img, eps=0.5, max_iter=100), không dùng Otsu.
- Lưu lịch sử mỗi vòng: T, μ1, μ2, Tnew, số pixel mỗi nhóm.
- Thử trên ma trận Bài 1 và một ảnh thực; in số vòng lặp, ngưỡng cuối và mask.
- Xử lý trường hợp một nhóm rỗng và giải thích tiêu chí dừng.

```
def iterative_threshold(img, eps=0.5, max_iter=100):
  x = img.astype(np.float64)
```

```
T = (x.min() + x.max()) / 2
history = []
for it in range(max_iter):
    g1, g2 = x[x <= T], x[x > T]
    # TODO: xử lý nhóm rỗng
    # TODO: tính Tnew, lưu history, kiểm tra hội tụ
    pass
return T, history
```

Bài 6. Otsu từ histogram và đối chiếu OpenCV

Theo trang 51–58: với ngưỡng k, $\omega 0 = P(k)$, $\omega 1 = 1 - P(k)$, $m(k) = \Sigma(i \cdot pi)$ đến k và $\mu T = \Sigma(i \cdot pi)$; $\sigma^2 B(k) = [\mu T \cdot \omega 0 - m(k)]^2 / [\omega 0 \cdot (1 - \omega 0)]$. Bỏ qua ngưỡng khiến mẫu số bằng 0.

- Tính histogram và xác suất pi bằng NumPy; triển khai otsu_manual(img) tìm k cực đại $\sigma^2 B$.
- Vẽ đồ thị $\sigma^2 B$ theo k, đánh dấu k tốt nhất; so sánh với cv2.THRESH_OTSU.
- Dùng ảnh coins và một ảnh bị giảm tương phản; so sánh ngưỡng, tỷ lệ foreground và kết quả trực quan.
- Giải thích trường hợp histogram một đỉnh và ảnh có chiếu sáng không đồng đều.

```
def otsu_manual(img):
    hist = np.bincount(img.ravel(), minlength=256).astype(np.float64)
    p = hist / hist.sum()
    w = np.cumsum(p)
    m = np.cumsum(np.arange(256) * p)
    mu = m[-1]
    # TODO: tính between-class variance an toàn với w=0 hoặc w=1
    # TODO: chọn ngưỡng, trả về T và mảng variance
    pass
```

Bài 7. Adaptive Thresholding và độ nhạy tham số

- Sử dụng ảnh page hoặc sudoku (chiếu sáng không đều). So sánh Global T=127, Otsu, Adaptive Mean, Adaptive Gaussian.
- Thử blockSize = 11, 21, 31 và C = 2, 8 (blockSize phải là số lẻ > 1).
- Giữ nguyên ảnh, quy ước mask và phương pháp để so sánh từng tham số; lập bảng tỷ lệ foreground, số thành phần nhiễu nhỏ nếu có.
- Phân tích hiện tượng chữ bị đứt, dính nét, mất nét mảnh và nền bị nhiễu.

```
def adaptive_compare(gray, block_size=11, C=2):
    # TODO: áp dụng cv2.ADAPTIVE_THRESH_MEAN_C
    # TODO: áp dụng cv2.ADAPTIVE_THRESH_GAUSSIAN_C
    # TODO: trả về 2 ảnh nhị phân
    pass
```

Bài 8. Tiền xử lý và đánh giá định lượng

- Chọn ảnh tài liệu có bóng hoặc nhiễu; thử GaussianBlur(5×5) và medianBlur(5) trước Otsu/Adaptive Gaussian.
- So sánh tối thiểu 4 pipeline: Otsu, Blur+Otsu, Adaptive, Blur+Adaptive.
- Nếu có ground truth mask: tính IoU, Precision, Recall và F1 cho foreground; thống nhất màu trắng = foreground.
- Nếu không có ground truth: chỉ thống kê tỷ lệ trắng, thành phần nhỏ, và nhận xét trực quan; KHÔNG coi các chỉ số này là độ chính xác.
- Đề xuất pipeline tốt nhất, nêu ít nhất một trường hợp thất bại.

```
def binary_metrics(pred, gt):
    pred = pred > 0
    gt = gt > 0
    # TODO: TP, FP, FN; IoU, Precision, Recall, F1
    # TODO: xử lý trường hợp mẫu số bằng 0
```

```
pass
```

Bài 9. Ứng dụng: số hóa tài liệu / hóa đơn

Tình huống: ảnh chụp trang giấy có bóng đổ hoặc nền không đồng đều. Mục tiêu: tạo bản đen-trắng rõ chữ để lưu trữ.

- Thiết kế pipeline: đọc ảnh → chuyển xám → lọc nhiễu (nếu cần) → chọn Otsu hoặc Adaptive → mask cuối.
- So sánh tối thiểu 3 cấu hình; giữ chữ màu đen trên nền trắng và giải thích việc dùng BINARY hay BINARY_INV.
- Hiển thị ảnh gốc, histogram, kết quả từng cấu hình; thống kê tỷ lệ pixel đen và thời gian xử lý.
- Xuất ảnh kết quả; báo cáo 3 lỗi thường gặp (mất dấu tiếng Việt, dính chữ, bóng nền).

Bài 10. Challenge: kiểm tra sản phẩm / đếm vật thể bằng ngưỡng

Tình huống: ảnh các đồng xu, hạt hoặc linh kiện nằm trên nền tương phản. Mục tiêu: phân đoạn đối tượng và ước lượng số lượng.

- Chọn ảnh coins hoặc ảnh tự chụp; so sánh Fixed Threshold, Otsu và Adaptive trên cùng ảnh.
- Làm sạch mask bằng morphology opening/closing (mở rộng ngoài bài giảng), giải thích kernel và số lần lặp.
- Dùng cv2.connectedComponentsWithStats đếm thành phần sau khi lọc theo diện tích; vẽ bounding box và số thứ tự.
- Báo cáo số đối tượng, diện tích trung bình, số đối tượng bị dính/chia sai; đánh giá trên ít nhất 2 ảnh.
- Challenge: chọn tham số tự động cho hai điều kiện sáng khác nhau, nêu lý do không có một thuật toán tốt nhất cho mọi ảnh.

V. CÂU HỎI

• Vì sao Otsu có thể hoạt động tốt trên histogram hai đỉnh nhưng kém trên ảnh chiếu sáng không đều?

• Tại sao Adaptive Gaussian và Adaptive Mean cho kết quả khác nhau dù cùng blockSize và C?

• Tại sao nên làm mờ trước khi tìm ngưỡng trên ảnh nhiễu? Khi nào làm mờ gây mất chi tiết?

• Ảnh nhị phân và ảnh nhãn đa mức khác nhau như thế nào?

• Nếu không có ground truth, có thể khẳng định phương pháp nào chính xác nhất không? Vì sao?

VI. TÀI LIỆU THAM KHẢO

1. Lecture 8 – Phân đoạn ảnh (PDF bài giảng do giảng viên cung cấp), trang 22–64.

2. OpenCV: Image Thresholding — https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

3. OpenCV: Otsu’s Binarization — https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

4. scikit-image: Sample Images — https://scikit-image.org/docs/stable/api/skimage.data.html

5. OpenCV sample images — https://github.com/opencv/opencv/tree/4.x/samples/data